EXAMEN PARCIAL 1
“Experimento Notación Big O”

Universidad Da Vinci de Guatemala
Facultad de Ingeniería, Industria y Tecnología
Curso: Estructura de Datos
Catedrático: Ing. Brandon Chitay

Wilson Oswaldo Vivar Solís
Carnet 202502391
03/03/2026

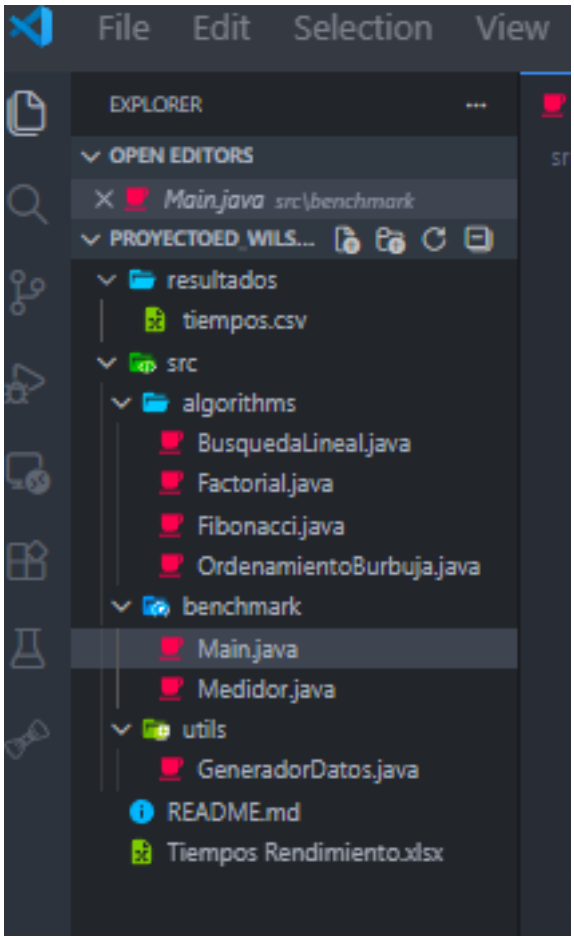
Análisis Big O

En el siguiente trabajo se realiza la experimentación de la medición de datos utilizando la notación Big O, sobre los algoritmos base de búsqueda y ordenamiento como lo son Factorial, Fibonacci, Búsqueda Lineal y Ordenamiento de Burbuja.

El fin de esto es conocer que algoritmo tiene un mejor desempeño a la hora de manejar cierta cantidad de datos y ver la tendencia que estos pueden llegar a tener de forma temporal y espacial en su ejecución, para así tomar una mejor decisión al momento de implementar código como desarrolladores de software.

Desarrollo del proceso:

1. Entorno de trabajo y estructura del proyecto



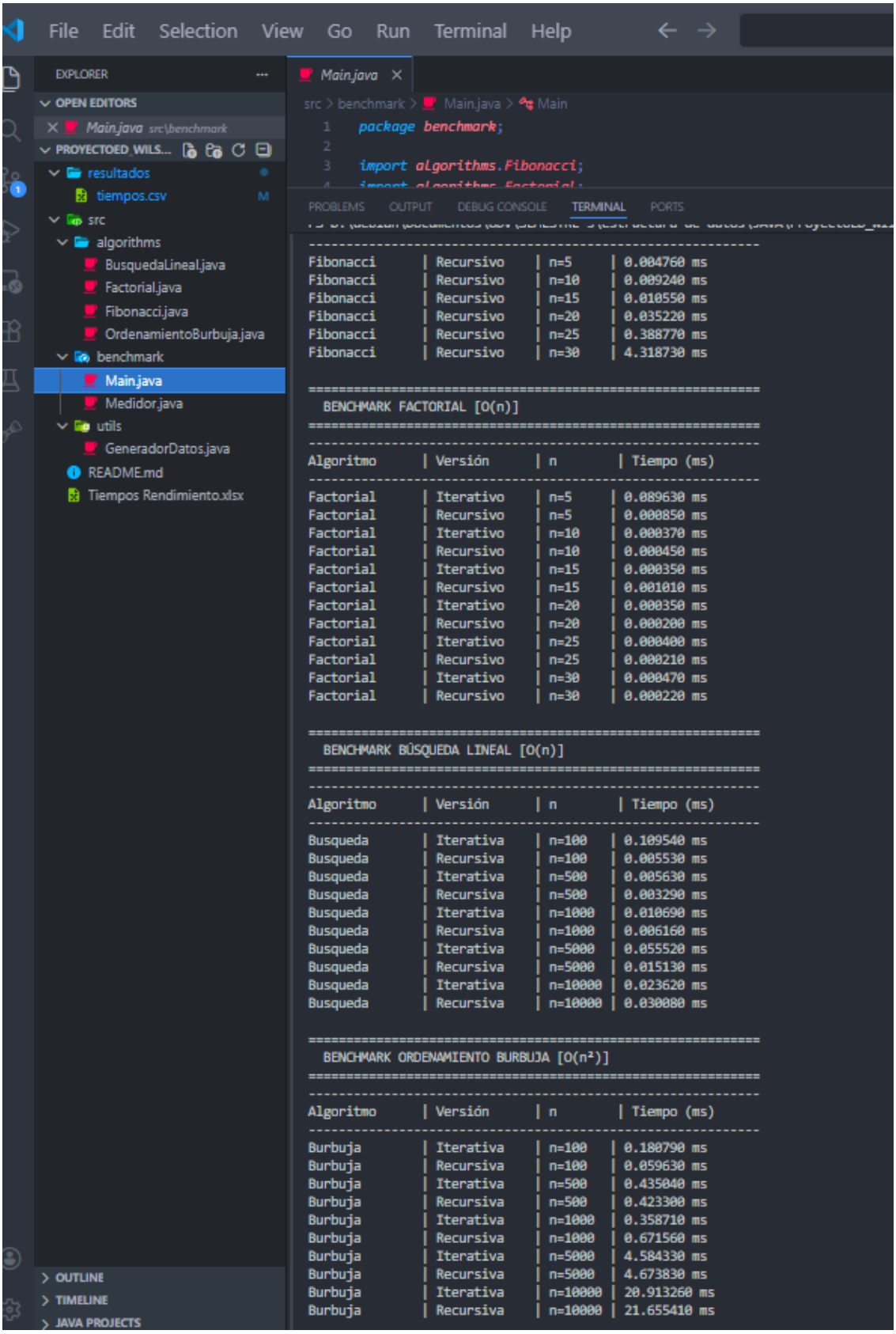
2. Vista general del Main.java

Main.java

D:\debian\Documentos\UDV\SEMESTRE 3\estructura de datos\JAVA\ProyectoED_WilsonVivar\src\benchmark\Main.java (preview)

```java
import algorithms.Fibonacci;
import algorithms.Factorial;
import algorithms.BusquedaLineal;
import algorithms.OrdenamientoBurbuja;
import utils.GeneradorDatos;

import java.io.FileWriter;
import java.io.IOException;
import java.io.PrintWriter;

/**
 * ============================================================
 * PUNTO DE ENTRADA — BENCHMARK FIBONACCI
 * ============================================================
 * Ejecuta ambas versiones de Fibonacci para distintos valores de n,
 * mide sus tiempos y exporta los resultados a un archivo CSV.
 *
 * CÓMO COMPILAR (desde la carpeta raíz del proyecto):
 * javac -d out src/algorithms/Fibonacci.java src/benchmark/Medidor.java
 * src/benchmark/Main.java
 *
 * CÓMO EJECUTAR:
 * java -cp out benchmark.Main
 * ============================================================
 */
public class Main {

    // ------------------------------------------------------------
    // TAMAÑOS DE PRUEBA
    // ------------------------------------------------------------
    /**
     * Valores de n para los que se ejecutará el benchmark.
     *
     * ITERATIVO: puede manejar n grandes (hasta ~92 con long)
     * RECURSIVO: limitado a n ≤ 30 por la complejidad O(2^n)
     * fib(30) → ~2.7 millones de llamadas
     * fib(40) → ~2.7 mil millones de llamadas (tarda minutos)
     */
    private static final int[] TAMANOS = { 5, 10, 15, 20, 25, 30 };
    /** Valores específicos para algoritmos de arreglos para notar la curva Big O */
    private static final int[] TAMANOS_GRANDES = { 100, 500, 1000, 5000, 10000 };

    /** Ruta del archivo de resultados */
    private static final String CSV_PATH = "resultados/tiempos.csv";

    // ------------------------------------------------------------
    // MAIN
    // ------------------------------------------------------------
    Run | Debug
    public static void main(String[] args) {
        imprimirBanner();

        StringBuilder csv = new StringBuilder();
        csv.append("Algoritmo,Version,n,Resultado,Tiempo_ms\n");

        // ---- FIBONACCI ITERATIVO ----
        System.out.println("\n  FIBONACCI ITERATIVO  [O(n)]");
        Medidor.imprimirEncabezado();

        for (int n : TAMANOS) {
            final int fn = n;

            // Calcular resultado una vez (solo para mostrarlo)
            long resultado = Fibonacci.iterativo(fn);

            // Medir solo el algoritmo puro (sin I/O ni inicialización)
            double tiempoMs = Medidor.medir(() -> Fibonacci.iterativo(fn));

            Medidor.imprimirFila("Fibonacci", "Iterativo", n, tiempoMs);
            csv.append(String.format("Fibonacci,Iterativo,%d,%d,%.6f%n", n, resultado, tiempoMs));
        }

```

3. Ejecución de los algoritmos



4. Vista general del Factorial.java

```
Factorial.java
src > algorithms > Factorial.java > {} algorithms
1  package algorithms;
2
3  /**
4   * ============================================================
5   * A3 - FACTORIAL
6   * ============================================================
7   * Calcula el producto de todos los números desde 1 hasta n.
8   * * Complejidad temporal (Ambas versiones):
9   * - O(n) — Se realizan exactamente n multiplicaciones.
10  *
11  * Complejidad espacial:
12  * - Iterativo: O(1) — Solo usa una variable acumuladora.
13  * - Recursivo: O(n) — Debido a la pila de llamadas (call stack).
14  * ============================================================
15  */
16 public class Factorial {
17
18     // VERSIÓN ITERATIVA [O(n)]
19     public static double iterativo(int n) {
20         if (n < 0) throw new IllegalArgumentException(s: "n no puede ser negativo");
21         double resultado = 1;
22         for (int i = 2; i <= n; i++) {
23             resultado *= i;
24         }
25         return resultado;
26     }
27
28     // VERSIÓN RECURSIVA [O(n)]
29     public static double recursivo(int n) {
30         if (n < 0) throw new IllegalArgumentException(s: "n no puede ser negativo");
31         // Caso base
32         if (n <= 1) return 1;
33         // Llamada recursiva
34         return n * recursivo(n - 1);
35     }
36 }
```

5. Vista general de BusquedaLineal.java

```
BusquedaLineal.java
src > algorithms > BusquedaLineal.java > {} algorithms
1  package algorithms;
2
3  public class BusquedaLineal {
4
5      // VERSIÓN ITERATIVA [O(n)]
6      public static int linealIterativa(int[] arreglo, int objetivo) {
7          for (int i = 0; i < arreglo.length; i++) {
8              if (arreglo[i] == objetivo) return i;
9          }
10         return -1;
11     }
12
13     // VERSIÓN RECURSIVA [O(n)]
14     public static int linealRecursiva(int[] arreglo, int objetivo, int indice) {
15         // Caso base: no se encontró el elemento
16         if (indice >= arreglo.length) return -1;
17
18         // Caso base: elemento encontrado
19         if (arreglo[indice] == objetivo) return indice;
20
21         // Llamada recursiva avanzando al siguiente índice
22         return linealRecursiva(arreglo, objetivo, indice + 1);
23     }
24 }
```

6. Vista general de OrdenamientoBurbuja.java

OrdenamientoBurbuja.java

src > algorithms > OrdenamientoBurbuja.java > { } algorithms

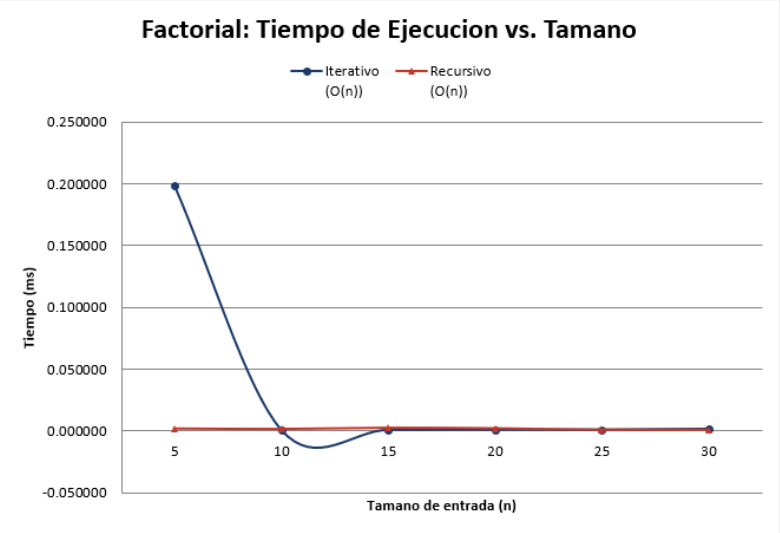
```java
package algorithms;

public class OrdenamientoBurbuja {

    // VERSIÓN ITERATIVA [O(n^2)]
    public static void burbujaIterativa(int[] arreglo) {
        int n = arreglo.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arreglo[j] > arreglo[j + 1]) {
                    int temp = arreglo[j];
                    arreglo[j] = arreglo[j + 1];
                    arreglo[j + 1] = temp;
                }
            }
        }
    }

    // VERSIÓN RECURSIVA [O(n^2)]
    public static void burbujaRecursiva(int[] arreglo, int n) {
        // Caso base: si el tamaño es 1, ya está ordenado
        if (n == 1) return;

        // Una pasada del burbuja (mueve el mayor al final)
        for (int i = 0; i < n - 1; i++) {
            if (arreglo[i] > arreglo[i + 1]) {
                int temp = arreglo[i];
                arreglo[i] = arreglo[i + 1];
                arreglo[i + 1] = temp;
            }
        }

        // Llamada recursiva para el resto del arreglo (n-1)
        burbujaRecursiva(arreglo, n - 1);
    }
}
```

7. Gráfica Factorial

HOJA — A1 - Factorial | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
5	0.199020	0.001876
10	0.000814	0.001442
15	0.000912	0.002628
20	0.001026	0.002114
25	0.001118	0.000614
30	0.001784	0.000714

Observacion: Ambas versiones son O(n): el tiempo crece linealmente con n.



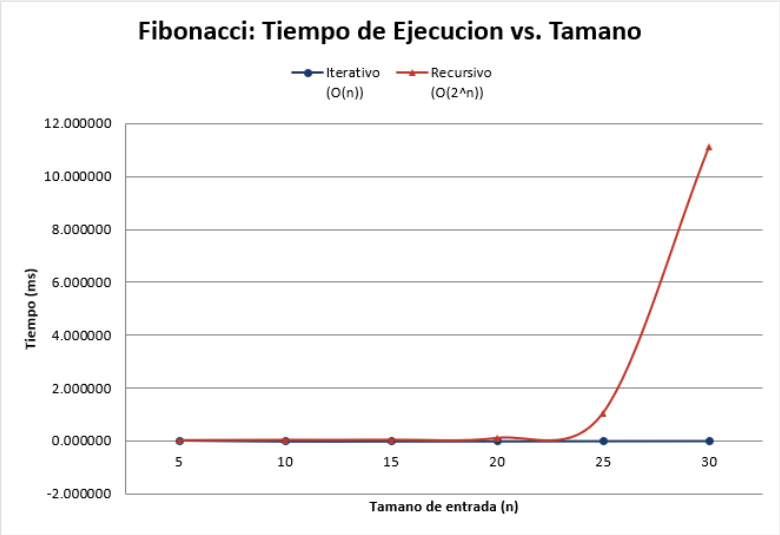
Se puede analizar que al inicio la línea Iterativa comienza con una carga de datos elevada, pero en el transcurso del tiempo se normaliza junto con la línea Recursiva. Ambas trabajan bien y se mantienen estables, haciendo notar que es un algoritmo con un uso de recursos del sistema aceptable.

8. Gráfica Fibonacci

HOJA — A2 - Fibonacci | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(2^n))
5	0.002620	0.009060
10	0.000786	0.021156
15	0.000896	0.026470
20	0.001030	0.093958
25	0.001322	1.041426
30	0.001504	11.115020

Observacion: Recursivo crece exponencialmente: duplicar n casi cuadriplica el tiempo.



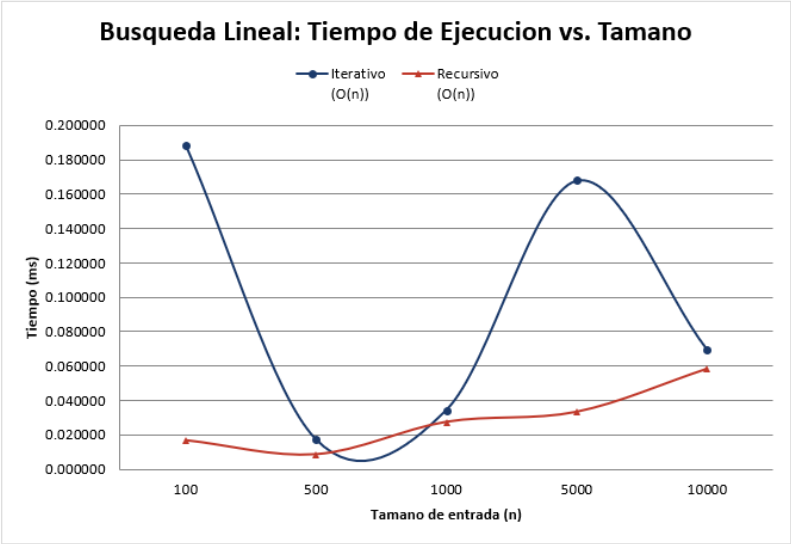
Para Fibonacci la línea estable y de rendimiento aceptable es la Iterativa, la cual refleja una estabilidad a lo largo del tiempo. Mientras que la línea recursiva se dispara hacia arriba manejo de datos más cargados, entonces se desaconseja su implementación en ciertos escenarios de carga alta de datos.

9. Gráfica Búsqueda Lineal

HOJA — A3 - Busqueda Lineal | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n)	Big-O Recursivo:

n	Iterativo (O(n))	Recursivo (O(n))
100	0.188722	0.016764
500	0.017204	0.008634
1000	0.034330	0.027462
5000	0.168008	0.033278
10000	0.069846	0.058224

Observacion: Ambas O(n): al multiplicar n x10, el tiempo tambien x10 aprox.



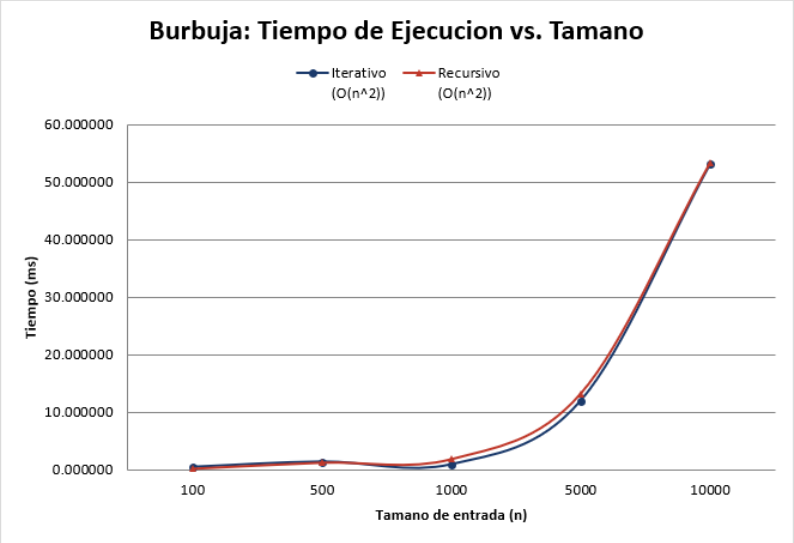
La Búsqueda Lineal, a mi parecer es ha sido una gráfica de variaciones importantes al ejecutar la carga de datos sugerida ya que la línea Iterativa forma varias variaciones a lo largo del tiempo hasta y la línea de Recursividad forma un incremento moderadamente suave y creciente que considero es más optimo su uso e implementación con ciertas cargas de datos.

10. Gráfica Ordenamiento de Burbuja

HOJA — A4 - Burbuja | Iterativo vs. Recursivo		
Big-O Iterativo:	O(n^2)	Big-O Recursivo:

n	Iterativo (O(n^2))	Recursivo (O(n^2))
100	0.428234	0.150724
500	1.348000	1.203978
1000	0.886200	1.820764
5000	11.976500	13.226722
10000	53.146852	53.313416

Observacion: Ambas O(n^2): al x10 n, el tiempo aumenta aprox x100.



La gráfica que más consideraciones se debe tener es la de Burbuja, ya que esta refleja incrementos considerables en un cierto incremento de datos para ambas líneas Iterativas y Recursivas, que si no son controlados pueden ocasionar un “StackOverflow” o llegar a un “Límite de memoria”.

11. Tabla de conclusiones

HOJA 6 — Conclusiones | Big-O Experimental vs. Teorico

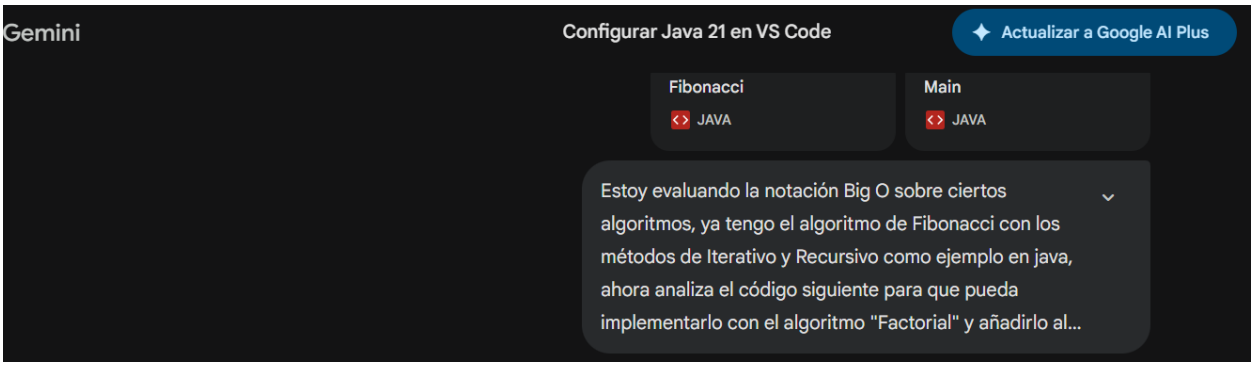
Algoritmo	Version	Big-O Experimental (de tu grafica)	Big-O Teorico	Coincide?	Justificacion breve
A1 - Factorial	Iterativo	O(1)	O(n)	Si / No	Si coincide ya que la grafica muestra lineas rectas.
A1 - Factorial	Recursivo	O(1)	O(n)	Si / No	Si coincide ya que la grafica muestra lineas rectas.
A2 - Fibonacci	Iterativo	O(1)	O(n)	Si / No	Si coincide ya que la grafica muestra lineas rectas.
A2 - Fibonacci	Recursivo	O(n^2)	O(2^n)	Si / No	Si, la linea se dispara de forma ascendente.
A3 - Busqueda Lineal	Iterativo	O(n)	O(n)	Si / No	si, la linea crece suavemente.
A3 - Busqueda Lineal	Recursivo	O(n log n)	O(n)	Si / No	No, en esta la linea Recursiva es más creciente.
A4 - Burbuja	Iterativo	O(n^2)	O(n^2)	Si / No	Si, la linea se dispara de forma creciente ascendente.
A4 - Burbuja	Recursivo	O(n^2)	O(n^2)	Si / No	Si, la linea se dispara de forma creciente ascendente.

Guia para interpretar tus graficas:	
Linea recta en grafica	→ Big-O es probablemente O(1) o O(n)
Curva suave ascendente	→ Big-O es probablemente O(n) o O(log n)
Curva que sube rapido	→ Big-O es probablemente O(n^2)
Curva que explota (vertical)	→ Big-O es probablemente O(2^n)

Mis conclusiones sobre el experimento del uso de “Notación Big O”, al darme cuenta sobre las cargas y llamadas de datos sobre los diferentes algoritmos implementados, pude diferenciar cual algoritmo es más estable y de rendimiento aceptable para procesar datos.

La utilización de los algoritmos en forma “Iterativa” son superiores en el manejo de memoria y tiempos de respuesta aceptables actualmente.

Más a detalle en los algoritmos de Búsqueda y Burbuja, en su uso de procesamiento por “Recursión” no son escalables para grandes volúmenes de datos, y pueden agotar o desbordar el uso de la memoria si no se tiene cuidado como implementadores o programadores.

Nota:

Para poder desarrollar los demás algoritmos (Factorial, Búsqueda Lineal y Ordenamiento de Burbuja) se incorpora la ayuda de la IA de Gemini, con el envío de “prompts” de solicitud para la creación de esos algoritmos para la evaluación del proceso de uso de la notación Big O.